Adaptive Learning Intelligence Platform: Retrieval-Augmented Study Assistant over a Student's Own Documents

A Dhanush
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY]
Roll No. 23K91A0501
[email address]

Ale Vijayalaxmi
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY]
Roll No. 23K91A0508
[email address]

Burri Mallikarjun
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY]
Roll No. 23K91A0549
[email address]

Atragada Nagatrimurthulu
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY]
Roll No. 23K91A0517
[email address]

Raghavendra
[DESIGNATION], Project Guide
Dept. of Computer Science and Engineering
TKR College of Engineering & Technology
[CITY]
[email address]

***Abstract*---**Students keep syllabi, lecture notes and question banks as scattered files that learning portals store but do not read, while general chatbots answer from public text instead of the prescribed syllabus. This paper presents the Adaptive Learning Intelligence Platform, a web application that extracts subjects and topics from uploaded PDF, DOCX and TXT files, answers questions from the student's own material with the source passage shown, generates fill-in-the-blank quizzes and ranks topics by quiz accuracy in a study roadmap. Retrieval is TF-IDF with cosine similarity and answers are extractive sentences; optional LLM generation exists but was not measured. On a six-subject notes corpus of 100,444 words with 72 hand-written questions, the shipped retriever places the evidence in its top four chunks for 79.2% of questions, while BM25 combined with dense embeddings reaches 88.9%. Extractive selection removes 87.5 % of the retrieved words but lowers answer-key recall from 0.861 to 0.667. Question-type routing and learned compression from the base paper are not implemented.

***Index Terms*---**context compression, educational documents, learning analytics, personalised roadmap, question answering, retrieval-augmented generation, TF-IDF.

I. INTRODUCTION

A student handles scattered PDFs, syllabi, lecture notes and question banks across six to eight subjects every semester. Conventional e-learning portals store these files but do not understand what is inside them, and large language models explain concepts well but may answer from memory rather than from the prescribed source. Retrieval-augmented generation (RAG) retrieves the relevant passage first and answers from it [2], which suits a student who needs an answer that can be traced back to their own notes.

Today a student searches several files by hand, or pastes text into a chatbot with no guarantee that the reply comes from the syllabus. Recommendations are identical for every learner, ignore earlier performance, and nothing shows which topics have been studied or how well they are understood. The cost is time lost searching, answers that cannot be checked, and revision that is not aimed at the weak topics.

The base paper, SAC-RAG [1], improves RAG in two ways: it routes each question to a retrieval strategy by type, and it compresses the retrieved context with an LLM, cutting token use by 38-58 % for a 1-2 point drop in exact match on multi-hop question-answering benchmarks. It does not address a student's own documents, study planning or progress analytics, and other educational RAG systems [3], [4] target personalised tutoring or essay assessment rather than document-grounded study over a personal curriculum. This work builds that missing combination and measures it honestly, including where it falls short of the base paper.

The contributions are: (1) a document-analysis module that stores uploaded study files as chunks with detected subjects, units and topics; (2) a question-answering engine that answers from the student's own chunks with source file, passage and match score, and abstains when nothing overlaps; (3) cloze quizzes whose per-topic results drive a priority-ranked study roadmap and dashboard behind a FastAPI service and a React interface; and (4) a measurement on a six-subject corpus of retrieval, extractive compression, abstention, quiz quality, latency and 84 automated tests, with what was not measured stated. Section II reviews related work, Section III gives the method, Section IV the results and Section V concludes.

A. System Hardware

All measurements ran on one desktop: Intel Core i5-12500 (6 cores, 12 threads), 15.7 GB RAM, none (CPU only), Windows 11 Pro. Indexing 672 chunks takes about 0.20 s and a query about 2.0 ms on this machine.

B. Software Requirements

Python 3.11.9, scikit-learn 1.5.1 [5], NumPy 1.26.4, pypdf 4.3.1 and python-docx 1.1.2 for parsing, FastAPI 0.115.0 with uvicorn 0.30.6, Flask 3.0.3 for the original interface, SQLite for storage, and React 19 with Vite 8.3 and Tailwind CSS 4 on Node 24.19.0 for the browser interface. A dense BGE-small model [6] was run only as a measurement reference.

II. LITERATURE SURVEY

Base paper. SAC-RAG [1] observes that standard RAG applies one retrieval strategy to every question and passes

redundant chunks to the generator. An LLM classifier labels each question Fact, Definition or Reasoning (92.0 % agreement with manual labels on 300 questions). Fact questions use hybrid BM25 plus dense retrieval with a cross-encoder reranker, Definition questions use dense retrieval only, and Reasoning questions are decomposed into sub-questions retrieved in parallel; a unified LLM module then extracts evidence snippets before generation. On HotpotQA, 2WikiMultiHopQA and MuSiQue, with 1,000 questions each and gpt-5-mini, adaptive retrieval raises HotpotQA exact match from 0.560 to 0.611 and Recall@10 from 0.787 to 0.880, and compression cuts token use by 38-58 % at a cost of 1-2 points of exact match. The price is latency: about 27.8 s per query against 9.6 s for naive RAG, because of the extra LLM calls.

Foundations. RAG was introduced by Lewis et al. [2]; dense passage retrieval [7] maps questions and passages into a shared vector space, while lexical scoring remains competitive: TF-IDF weighting [8] and the BM25 probabilistic framework [9] reward terms that are frequent in a passage and rare in the collection. Latent semantic analysis [10] gives a low-dimensional lexical-semantic compromise, and general-purpose embedding models such as BGE [6] are the usual dense choice. HotpotQA [11] is the benchmark family used by [1].

Adaptive retrieval and compression. Adaptive-RAG [12] routes questions among no retrieval, single-step and multi-step retrieval by complexity, and Self-RAG [13] lets the model decide when to retrieve; both are described in [1] as lacking a compression stage. Long contexts are used less well by LLMs, which motivates compression [14]: LLMLingua [15] prunes prompts with a small language model and RECOMP [16] casts compression as summarisation, but both sit on top of a fixed retrieval strategy [1].

Educational RAG. TutorLLM [3] combines knowledge tracing [17] with RAG so that answers follow the learner's estimated state, and reports about 10 % higher user satisfaction and 5 % higher quiz scores than a general LLM. An LLM assessment RAG [4] retrieves rubrics and exemplar essays to grade and comment on essays, with 94-99 % agreement with human evaluators on 701 essays. Neither answers questions from a student's own uploaded notes or ranks the student's topics by measured weakness.

TABLE I
SELECTED RELATED WORK

Work	Approach	Data	Reported result	Limitation
SAC-RAG [1]	LLM question typing, type-specific retrieval, LLM compression	HotpotQA, 2Wiki, MuSiQue; 1,000 questions each	EM 0.560 to 0.611; tokens -38 to -58 %	Extra LLM calls (27.8 s vs 9.6 s)
TutorLLM [3]	Knowledge tracing plus RAG with a web scraper	Learner study with questionnaires	+10 % satisfaction, +5 % quiz score	Needs learner interaction history
Assessment RAG [4]	LLM agent retrieving rubrics and exemplar essays	701 student essays	94-99 % agreement with humans	Grades essays; no study-note QA
Adaptive-RAG [12]	Routes by question complexity	(see [1])	Not read in full	No compression stage [1]
LLMLingua [15]	Prompt compression by a small LM	(see [1])	Not read in full	Fixed retrieval strategy [1]
RECOMP [16]	Compression as summarisation	(see [1])	Not read in full	Fixed retrieval strategy [1]

Research gap. The surveyed work optimises retrieval or compression for general questions [1], personalises tutoring [3] or grades essays [4]. None combines ingestion of a student's own syllabus and notes, answers that cite the source passage, and progress analytics that rank topics by measured weakness. We build that combination and, since it is a small system without an LLM in the measured path, we report how far its retrieval and compression fall short of the base paper's.

III. METHODOLOGY

Pipeline. Fig. 1 shows the layers and Fig. 2 the main flow. On upload the text is extracted with pypdf or python-docx, whitespace is normalised and the text is cut into overlapping windows of 180 words with a 30-word overlap. Lines such as "Subject:", "Course:", "Unit 3:" and "Chapter 2:" are detected with regular expressions, and TF-IDF keywords over sentences fill the remaining places of a list of at most 12 topics. Document, topics and chunks are stored in SQLite and the user's index is rebuilt.

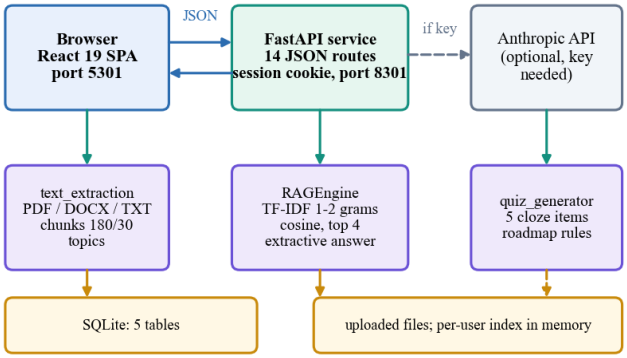


Fig. 1. System architecture: React front end, FastAPI service, shared Python modules, SQLite storage and an optional LLM.

Data. The product has no training set, so a six-subject notes corpus was built for measurement: 36 English Wikipedia articles [18] pinned to revision identifiers, grouped as Data Structures, Operating Systems, Database Systems, Computer Networks, Machine Learning and Algorithms, each file laid out with "Subject:" and "Unit n:" headings. It holds 100,444 words in 672 chunks. We wrote 72 questions, 12 per subject and 24 each of fact, definition and reasoning type (the base paper's taxonomy), with a gold evidence sentence copied from the text, plus 12 off-topic questions. A retrieved chunk is a hit when it contains the evidence.

Evaluation protocol. Every evidence string was checked to occur exactly once in the corpus and wholly inside at least one chunk (72 of 72), so a miss is a retrieval miss and not a cut made by the chunker at the shipped size. The answer key

is a short string with one to three accepted alternatives, searched in the retrieved text and in the generated answer; it is deliberately lenient, so it measures whether the needed fact survives and not whether the answer reads well. Confidence intervals come from 2,000 bootstrap resamples of the questions. The off-topic set (football, cooking, geography and similar) tests abstention. Quiz quality is checked automatically on 900 questions (30 seeds, six files, five questions) and by one reviewer's labels on 30; latency is measured in process and over HTTP on 127.0.0.1 with 30 calls per endpoint.

Retrieval. Each chunk and the query are mapped to TF-IDF vectors over unigrams and bigrams with English stop words [5], [8], and chunks are ranked by cosine similarity,

$$s(q, c) = (q \cdot c) / (|q|\ |c|), \qquad (1)$$

keeping the top k = 4 chunks with s > 0. TF-IDF was chosen because it needs no model download, runs on any laptop and gives a score that can be shown to the student. For comparison we also evaluate BM25 [9] with k1 = 1.5 and b = 0.75,

$$BM25(q,d) = \Sigma\ IDF(w)\ f(w,d)(k1+1) / (f(w,d) + k1(1 - b + b|d|/avgdl)), \qquad (2)$$

latent semantic analysis with 200 dimensions [10], dense BGE-small embeddings [6], and min-max-normalised 50/50 hybrids; only the TF-IDF retriever is part of the product. Quality is measured by the hit rate and the mean reciprocal rank over a question set Q,

$$Hit@k = (1/|Q|)\ \Sigma\ 1[\text{evidence in top-}k], \qquad (3)$$

$$MRR = (1/|Q|)\ \Sigma\ 1/rank. \qquad (4)$$

Answering. The default answer is extractive: the retrieved chunks are split into sentences longer than 20 characters, ranked by TF-IDF cosine against the question, and the top four with a positive score are joined. If an API key is configured the chunks and question go to an LLM instructed to answer only from the context, and any failure falls back to the extractive answer. The words removed are measured by

$$compression = 1 - |answer| / |retrieved\ context|. \qquad (5)$$

Knowledge repository. Each upload creates one document row (file name, subject, topic list) and one row per chunk, linked by foreign keys that cascade on deletion, so removing a document removes its chunks and its file. The topic list puts the detected headings first, for example the subject line and the unit titles, and fills the rest with the highest-weighted unigrams and bigrams of the document's sentences. The retrieval index is held in memory per user and rebuilt after every upload or deletion; rebuilding is linear in the number of chunks, which is acceptable for a personal collection but would need incremental updates for a shared one.

Quiz and roadmap. A quiz blanks the first occurrence of a high-weight TF-IDF term in a sentence and offers three other terms as distractors. A topic's accuracy is 100 x correct / attempts over its quiz rows and its priority is High below 50, Medium below 80 and Low otherwise. **Implementation.** The service exposes 14 endpoints with a signed session cookie and Pydantic bodies; the React pages are code-split. The suite has 84 pytest tests and a 24-check smoke test.

Security and error handling. Passwords are stored as salted hashes and the session cookie is signed, HttpOnly and SameSite=Lax; every SQL statement is parameterised; file names pass through a sanitiser and only PDF, DOCX and TXT files up to 16 MB are accepted; every document query filters on the user's identifier, so one user cannot list, delete, quiz on or search another's files. The tests assert these behaviours: unauthenticated calls to nine protected routes return 401, SQL-injection strings are stored as literal text, and a file named "../../evil.txt" is saved as "evil.txt" inside the user's folder. They also assert two known weaknesses: two uploads with the same name share one path on disk, and the cookie is not marked Secure in the local deployment.

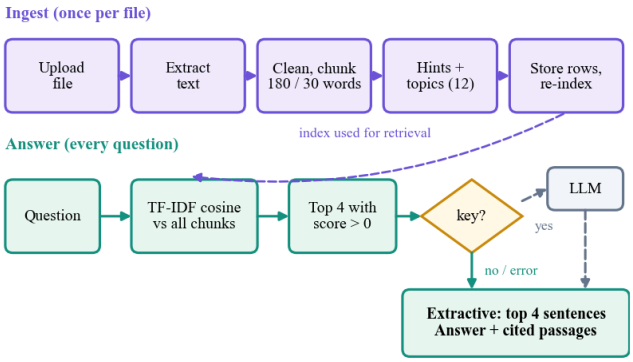


Fig. 2. Main flow: ingestion of a document and answering of a question.

IV. RESULTS

Test conditions. All 72 questions are test questions; nothing is trained. The index holds 672 chunks of the six files in one user account, as in the application. The bootstrap 95 % interval of the shipped retriever's Hit@4 is 0.694-0.875, so differences of a few points between retrievers are within noise. Table II gives the retrieval results.

TABLE II
RETRIEVAL ON 672 CHUNKS, 72 QUESTIONS

Retriever	Hit@1	Hit@4	Hit@10	MRR@10
TF-IDF 1-2 gram (shipped)	0.403	0.792	0.931	0.588
TF-IDF unigram	0.389	0.778	0.944	0.577
BM25	0.500	0.833	0.944	0.659
TF-IDF + LSA (200)	0.431	0.764	0.958	0.585
BM25 + LSA	0.500	0.833	0.944	0.658
Dense BGE-small	0.431	0.778	0.917	0.588
BM25 + dense BGE-small	0.556	0.889	0.958	0.709

Retrieval (Fig. 3a). The shipped retriever reaches Hit@1 0.403, Hit@4 0.792 and MRR 0.588. BM25 is better at the first position (0.500) and the BM25-plus-dense hybrid is best on every column (Hit@4 0.889), so the "semantic" retrieval named in the objectives is not delivered by TF-IDF. By type, Hit@4 is 0.833 for fact, 0.833 for definition and 0.708 for reasoning questions; at rank one, definition questions are weakest (0.292). Larger chunks help the measure: Hit@4 rises from 0.708 at 100 words to 0.847 at 300, but they hand more text to the reader, so 180 was kept.

By question type. The base paper routes fact, definition and reasoning questions differently, so we split the results the same way. TF-IDF scores Hit@4 0.833, 0.833 and 0.708; BM25 scores 0.958, 0.750 and 0.792; the BM25-plus-dense hybrid 0.958, 0.875 and 0.833. No single retriever wins every type, which is consistent with the base paper's argument for type-specific strategies, but with 24 questions per type these gaps are suggestive, not conclusive, and we did not implement a router to test it.

Compression (Fig. 3b). With four chunks retrieved, the extractive step turns 720 words into 90 (87.5 % removed), and every answer word comes from the retrieved chunks. The answer keeps the key for 66.7% of questions where the full chunks held it for 86.1%, a loss of 19.4 points; for definition questions the key is in the chunks for 91.7% but in the answer for only 50.0%, because the sentences chosen overlap the question's words rather than state the definition.

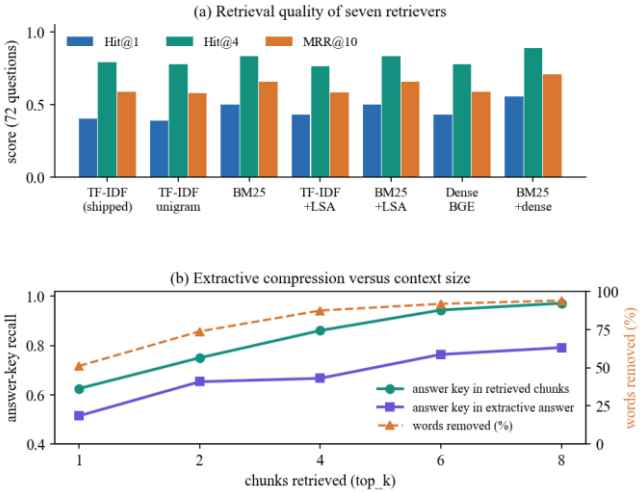


Fig. 3. (a) Retrieval results; (b) answer-key recall and words removed by the extractive answer step.

TABLE III
EXTRACTIVE ANSWER STEP BY CHUNKS RETRIEVED

k	Words in	Words out	Removed	Key in chunks	Key in answer
1	180	88	51.1 %	0.625	0.514
2	360	94	73.8 %	0.750	0.653
4	720	90	87.5 %	0.861	0.667
6	1080	87	91.9 %	0.944	0.764
8	1438	84	94.1 %	0.972	0.792

Even with eight chunks the answer step keeps the key in only 79% of answers where the chunks held it in 97%: the loss comes from sentence selection, not from retrieval. Given that retrieval succeeded, the answer holds the key for 82.5% of the 57 questions.

Chunk size and cost. Hit@4 is 0.611 at 60 words, 0.708 at 100, 0.792 at 180 (shipped), 0.847 at 300 and 0.944 at 450; the shortest windows also cut 2 of the 72 evidence sentences in two. The curve rewards larger chunks because fewer, longer candidates are easier to hit, so the choice trades recall against the amount of text handed to the reader. Dense retrieval needed 52 s to embed the 672 chunks on the CPU and 4.4 ms per query, against 197 ms to build the TF-IDF index, which is why it was kept out of the product.

Abstention. The engine refuses only when no query term occurs in any chunk, so 9 of 12 off-topic questions still received an answer. In-scope top-1 scores have a median of 0.32, off-topic ones 0.09; a threshold of 0.15 would refuse 83% of off-topic questions and answer 96% of in-scope ones (suggested, not implemented).

Roadmap and dashboard. For the screens we uploaded five documents, asked four questions and took eight quizzes with a scripted learner who knew a chosen number of answers per quiz, giving subject accuracies of 85 % (Data Structures), 80 % (Algorithms), 60 % (Operating Systems) and 30 % (Database Systems). The dashboard shows these as bars coloured by band, and the roadmap lists 30 topics: 9 High, 2 Medium and 19 Low (Fig. 5). The scripted answers show that the bands follow the accuracy thresholds, not that the roadmap improves learning: no user study was run, and the topic names are keywords, so a student would see generic entries such as "using" next to meaningful ones.

Analysis, quiz and speed (Fig. 7). All 36 "Unit n:" headings were found, but "UNIT-I", "Module 1" and "1." styles were found in 0 of 5 units, and text in DOCX tables is not read. Of 900 generated quiz questions all are well formed, yet the answer term remains visible in the stem for 18.2%, and a manual review rated 17 of 30 acceptable. Retrieval time grows about linearly from 2.0 ms at 672 chunks to 17.6 ms at 5,376; over HTTP an ask call has a median of 30 ms and login 128 ms (password hashing). 84 pytest tests and 24 smoke checks pass; the LLM answer mode, concurrent users and real student files were not measured.

Tests and builds. The 84 pytest tests (API 45, text extraction 15, security 11, retrieval engine 9, quiz generator 4) run in 15 s against a temporary database and pin the roadmap thresholds: 0, 33 and 49 % give High, 50, 78 and 79 % give Medium, 80 and 100 % give Low. The type-checked production build of the React interface succeeds, and a scripted browser walk through every screen at 1440, 768 and 360 pixels in light and dark themes found no console errors, page errors, failed requests or sideways scrolling.

Response times (Fig. 6). Over HTTP on one machine the medians are 4 ms for the dashboard, 6 ms for the document list, 7 ms for the roadmap, 23 ms to generate and 24 ms to submit a quiz. Uploading a 90-110 kB file re-indexes every chunk of the user and took between 0.12 and 2.5 s over two runs, the spread coming from other jobs on the machine; one or two of 30 quiz submissions took one to two seconds, probably a slow database commit, which we did not isolate.

Threats to validity. The questions were written by the team who knew the corpus, which may favour lexical overlap; Wikipedia prose is more regular than student notes, which are often terse or scanned; the evidence test is strict and can under-count chunks that state the fact in other words, while the answer-key test is lenient and can over-count; 72 questions give wide intervals; all numbers come from one machine; and the learner behind the roadmap screens was

scripted. The LLM mode, which the base paper's pipeline relies on, was not measured.

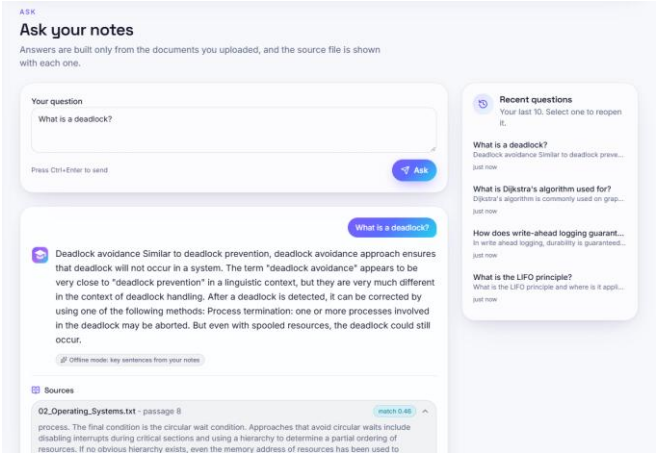


Fig. 4. Output screen: a question answered from the uploaded notes, with the source passage and match score.

Comparison with the base paper. The base paper evaluates multi-hop benchmarks with an LLM and reports exact match, F1, Recall@10 and tokens; we evaluate a 72-question notes corpus without an LLM and report hit rates and answer-key containment, so the numbers are not comparable. The pattern is: the base paper trades 1-2 points of exact match for 38-58 % fewer tokens, whereas our extractive step trades 19.4 points of key recall for 87.5 % fewer words. Its use of a BM25-plus-dense hybrid for fact questions is consistent with our data (hybrid retrieval was best, and BM25 beat TF-IDF on fact questions, Hit@4 0.958 against 0.833); we did not test question-type routing or LLM compression.

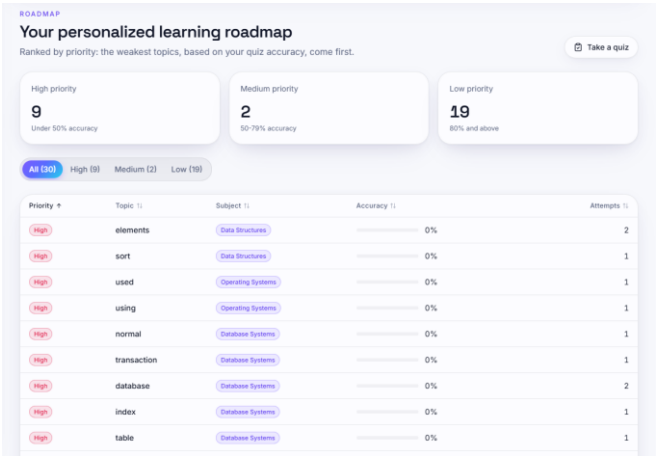


Fig. 5. Output screen: the study roadmap ranks topics by quiz accuracy; topics are single keywords, some generic.

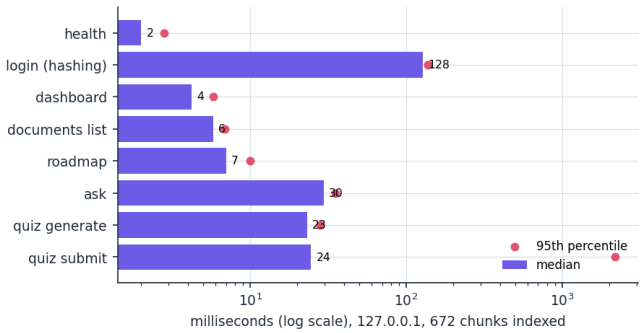


Fig. 6. Median (bar) and 95th-percentile (dot) latency of the API over HTTP on 127.0.0.1 with 672 chunks indexed.

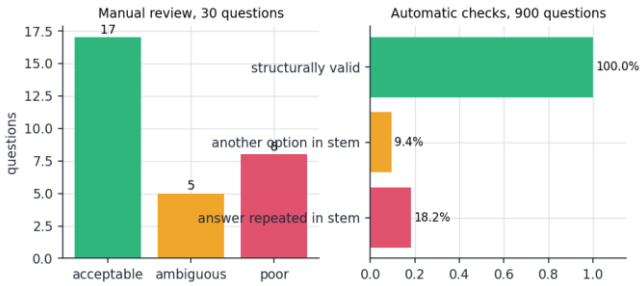


Fig. 7. Quiz quality: manual review of 30 questions (left) and automatic checks on 900 questions (right).

Limits and failure cases. Retrieval misses 21% of questions at four chunks; the extractive answer can place the right sentence outside its four; roadmap topics are keywords such as "using" or "normal"; and the roadmap uses only quiz accuracy, with no revision schedule, coverage or velocity measure. In Fig. 4 the question "What is a deadlock?" is answered from a passage with match score 0.46, but the sentences returned discuss deadlock avoidance and recovery rather than the definition, the failure the by-type results predict. Quiz sentences also sometimes begin in the middle of a heading (for example a blank before "2: Stacks and Queues"), and the same keyword can appear under two subjects in the roadmap.

V. CONCLUSION

The platform delivers document ingestion, source-cited answers, quizzes, a priority roadmap and a dashboard behind a tested API. Against the objectives, ingestion and cited answering are met, while embeddings, learned context compression and adaptive routing from the base paper are not implemented, and the roadmap covers quiz accuracy only. The central finding is that plain TF-IDF finds the evidence in the top four chunks for 79.2% of questions, BM25 plus dense embeddings for 88.9%, and that extractive compression removes 87.5 % of the words at a clear cost in answer-key recall.

Three observations follow. First, the lexical retriever is adequate for fact and definition lookups when the question reuses the notes' wording but weak at rank one, so showing four sources with match scores is part of the design. Second, compression by sentence selection, not retrieval, limits answer quality: the evidence was in the chunks for 86% of questions but in the answer for 67%. Third, with no relevance

threshold the system can answer an off-topic question from unrelated notes, which weakens the promise of answering strictly from the student's material.

Against the six objectives: ingestion that extracts subjects, units and topics is partly met (it needs headings in the supported forms); semantic embeddings are not met by the shipped retriever; answering from the uploaded material with its source is met in extractive mode; context compression is partly met by sentence selection; the roadmap is partly met as an accuracy ranking without a revision schedule; and the dashboard is partly met, showing counts and subject accuracy but no coverage or study velocity.

Limitations: one corpus of encyclopaedia text rather than student files, 72 questions, an unmeasured LLM mode, no relevance threshold, and process-local quiz state and index cache. Future work: adopt BM25 and a dense hybrid, add a relevance threshold and a question-type router, compress with an LLM and measure it, recognise Roman-numeral and module headings, read DOCX tables and scanned PDFs, replace keyword topics with unit-level topics, add revision scheduling, and run a user study on real syllabi.

REFERENCES

[1] D. Zhang, H. Yu, J. Huo and M. Su, "SAC-RAG: Semantic Adaptive Context Compression for Retrieval-Augmented Generation," IEEE Access, vol. 14, pp. 117406-117418, 2026, doi: 10.1109/ACCESS.2026.3718025.

[2] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 9459-9474.

[3] Z. Li, V. Yazdanpanah, J. Wang, W. Gu, L. Shi, A. I. Cristea, S. Kiden and S. Stein, "TutorLLM: Customizing learning recommendations with knowledge tracing and retrieval-augmented generation," in Proc. 18th ACM Conf. Recommender Syst. (RecSys 2024), Bari, Italy, Oct. 2024; arXiv:2502.15709.

[4] R. Vatankhah Barenji, N. Salimi and S. Khoshgoftar, "An LLM-powered assessment retrieval-augmented generation (RAG) for higher education," arXiv:2601.06141, Jan. 2026.

[5] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, Oct. 2011.

[6] S. Xiao, Z. Liu, P. Zhang and N. Muennighoff, "C-Pack: Packaged resources to advance general Chinese embedding," arXiv:2309.07597, 2023 (BGE embedding models, including BAAI/bge-small-en-v1.5).

[7] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen and W.-T. Yih, "Dense passage retrieval for open-domain question answering," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2020, pp. 6769-6781.

[8] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," Inf. Process. Manage., vol. 24, no. 5, pp. 513-523, 1988.

[9] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," Found. Trends Inf. Retr., vol. 3, no. 4, pp. 333-389, 2009.

[10] S. Deerwester, S. T. Dumais, G. W. Furnas, T. K. Landauer and R. Harshman, "Indexing by latent semantic analysis," J. Amer. Soc. Inf. Sci., vol. 41, no. 6, pp. 391-407, 1990.

[11] Z. Yang, P. Qi, S. Zhang, Y. Bengio, W. Cohen, R. Salakhutdinov and C. D. Manning, "HotpotQA: A dataset for diverse, explainable multi-hop question answering," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2018, pp. 2369-2380.

[12] S. Jeong, J. Baek, S. Cho, S. J. Hwang and J. Park, "Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity," in Proc. Conf. North Amer. Chapter Assoc. Comput. Linguistics: Hum. Lang. Technol. (NAACL-HLT), 2024, pp. 7036-7050.

[13] A. Asai, Z. Wu, Y. Wang, A. Sil and H. Hajishirzi, "Self-RAG: Learning to retrieve, generate, and critique through self-reflection," in Proc. 12th Int. Conf. Learn. Represent. (ICLR), 2024.

[14] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni and P. Liang, "Lost in the middle: How language models use long contexts," Trans. Assoc. Comput. Linguistics, vol. 12, pp. 157-173, Feb. 2024.

[15] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang and L. Qiu, "LLMLingua: Compressing prompts for accelerated inference of large language models," in Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP), 2023, pp. 13358-13376.

[16] F. Xu, W. Shi and E. Choi, "RECOMP: Improving retrieval-augmented LMs with compression and selective augmentation," in Proc. 12th Int. Conf. Learn. Represent. (ICLR), 2024.

[17] A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," User Model. User-Adapt. Interact., vol. 4, no. 4, pp. 253-278, 1994.

[18] Wikimedia Foundation, "English Wikipedia," https://en.wikipedia.org (text licensed CC BY-SA 4.0; the 36 articles and revision identifiers used for the evaluation corpus are listed in experiments/corpus_manifest.json).